

DIU CPC × upay AI Hackathon 2026

AgentEr Upay

7-day cash and e-float liquidity forecasting for upay MFS agents

Project report: approach, implementation, results and limitations

Prototype built on synthetic data. Recommendations only.

1. Summary

An upay agent holds two balances: **cash** and **e-float**. A cash-out moves value from cash to e-float, a cash-in does the reverse, and customers go unserved when either balance runs out. AgentEr Upay forecasts the next seven days of demand for each agent, converts the forecast into run-out risk, and tells people what to do: how much to top up and by when, with plain-language reasons.

The prototype has two roles. The **Agent app** is mobile-first (home gauges, a daily cash report, the 7-day forecast, history). The **Admin console** is desktop-first (risk-ranked network overview, agent detail with a demo reveal, dispatch plan and orders, rules and local events, alerts). It runs on a simulation clock that replays the 2025 synthetic dataset day by day, so a demo can show the nightly cycle: ingest, report, recompute, alert.

Item	Value
Agents / data	16 agents × 365 days (2025), synthetic
Model	Supplied LightGBM quantile bundle, used as-is (not retrained)
Backend	FastAPI, SQLite, pandas, lightgbm, JWT auth
Frontend	Next.js (App Router), TypeScript, Tailwind, Recharts, Framer Motion
Languages / themes	English and Bangla (digits follow the language); light and dark
Automated tests	45 backend tests passing; frontend lint and type-checked production build clean

2. The problem

upay agents are the last mile of mobile financial services. Every agent holds two balances at once: **cash** in the till and **e-float** in the upay wallet. A cash-out customer takes cash and the agent receives e-float; a cash-in customer does the opposite. Each transaction therefore drains one balance while filling the other, and a customer goes unserved the moment either one runs out.

Why this is hard

- **Demand is uneven and predictable only in part.** It swings with weekdays, payday, holidays and Eid, and differs by agent type: garment-area, university, rural, market and remittance-urban agents behave very differently.
- **Replenishment is slow and calendar-bound.** Topping up means a trip to a distributor or a bank, and distributor offices work only on working days (Fridays, Saturdays and holidays are closed). A stock-out that starts on a Thursday evening may not be fixable until the next working day.
- **Agents plan by habit.** The common rule of thumb is to reorder when a balance falls below about 30% of capacity and top up to about 65%. It ignores upcoming demand, so it is too cautious in quiet weeks and too late in busy ones.
- **Cash is self-reported.** The e-float and the transaction flows are in the ledger, but the physical cash count comes from the agent and may be late, missing or wrong.

- **Some agents are structurally under-capitalised.** For a few (garment areas in particular), the safe level of cash plus e-float is more than their total float, so better timing alone cannot fix it.

Who feels it

- **Agent:** asks "Will I run out, when, and what should I do today?" and needs the answer in seconds, on a phone, in Bangla or English.
- **Distributor/admin:** asks "Which agents need an order, how urgent, how big, and does anyone need more capital?" across the whole network.
- **Customer:** simply finds the agent unable to serve, which is a lost transaction and a loss of trust in upay.

3. Proposed idea

AgentEr Upay turns "I think I might run low" into a specific, explained, time-bound recommendation. The idea has three parts:

1. **Forecast the distribution, not one number.** Predict next seven days of cash-in and cash-out as ranges (P10 to P90), then simulate thousands of possible paths of the two balances to get the **probability of running out** of each, and the likely run-out date.
2. **Turn the risk into an action using business rules.** The risk is compared with the next day a top-up can actually arrive (the next working day), and the app recommends *how much* to top up and *by when*, whether it is a planned or an emergency order, and whether the agent needs extra capital because the float is simply too small.
3. **Explain it in plain language.** Each forecast shows the top reasons (for example payday, an Eid window, a weekday effect) in taka, so an agent or a distributor can see why, and challenge it.

Design principles

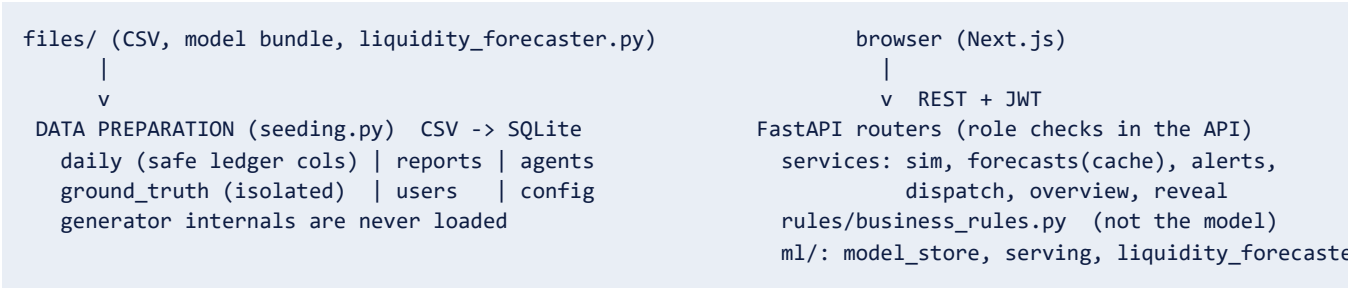
1. **Use the supplied model and serving code unchanged.** Feature building, the risk engine and the explanations are imported, never reimplemented. The model file is never modified.
2. **Keep the model and the business rules separate.** Buffers, coverage, thresholds, top-up sizing and timing, reconciliation and manual event overrides live in a rules module and a config table, so they can change without retraining.
3. **No ground truth in the product.** True demand columns exist only in an isolated table read by the admin "Demo reveal" overlay. Generator internals are never loaded.
4. **Recommendations only.** People decide. No language model produces any number or decision; explanations are deterministic templates built from model output.
5. **Show uncertainty honestly.** Demand as bands, risk as percentages, each value labelled with its source (model quantile or the rule that produced it).
6. **Two roles, two designs.** A mobile-first agent app and a desktop-first admin console, in English and Bangla, light and dark.

Method

The work was built in five verified phases: (1) backend, seed and forecast endpoint; (2) design system and agent app; (3) admin console; (4) rules, events, dispatch and alerts; (5) evaluation summary, polish, tests and documentation. After the first delivery the interface was reworked on request (see section 8).

4. Implemented solution

The solution is a working web application with a Python API and a replayed 2025 dataset, driven by a simulation clock so the whole daily cycle can be demonstrated: ingest the day, request the cash report, recompute forecasts, raise alerts.



Layer	Responsibility
seeding.py	Data preparation only: splits the CSV into safe ledger columns, reports, agents, users, config; isolates ground truth.
ml/model_store.py	Loads the bundle once at startup; warns on library version mismatch; hands out copies of the risk config.
ml/serving.py	Inference only: features, quantiles, optional human override, Monte Carlo risk, balance bands, top-5 drivers.
rules/business_rules.py	Status bands, reconciliation, top-up sizing and timing, capital gap, order type, event multipliers, rule validation.
services/	Simulation clock and nightly pipeline, forecast cache, alerts, dispatch plan, overview, demo reveal.
routers/	HTTP API, JWT auth, role enforcement, one error envelope.

The supplied `liquidity_forecaster.py` is copied byte-for-byte into the backend, and a test asserts it matches the original. The forecast cache is keyed by agent, date, config hash, cash input and event multipliers; thresholds are applied after the cache so editing them needs no re-simulation.

Backend

FastAPI with SQLAlchemy over SQLite. Library versions are pinned to those recorded in the model bundle (lightgbm 4.7.0, pandas 3.0.2, numpy 2.4.4). Authentication uses bcrypt-hashed mock users and JWTs. Agents can read only their own data, enforced in the API; admins can read everything. All errors share one envelope.

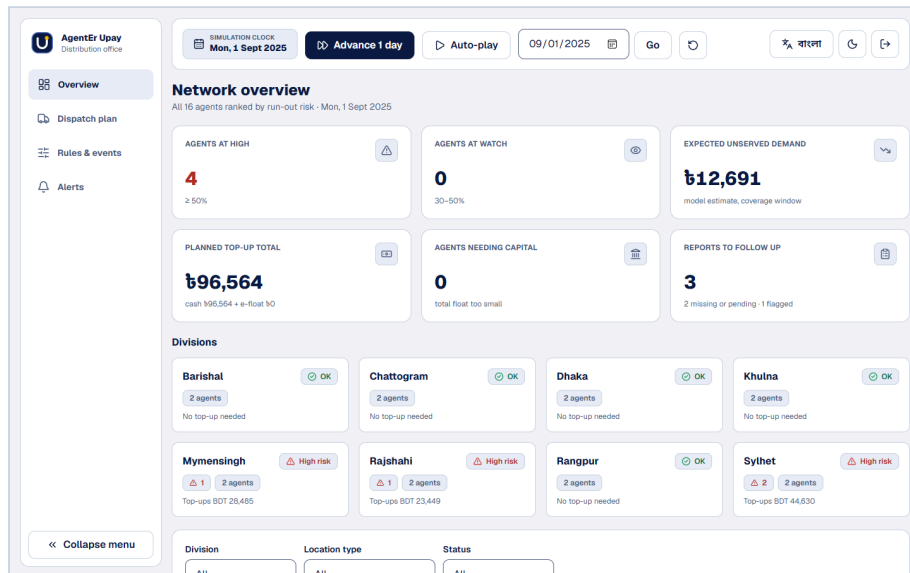
The model bundle is loaded once at startup. Forecasts are cached per agent, date, config and cash input, so one forecast takes about 40 ms (about 120 ms with explanations) and a cold 16-agent overview about 0.8 s.

Simulation clock and nightly pipeline

The clock runs from 2025-01-28 (28 days of history needed) to 2025-12-31 and starts at 2025-09-01. "Advance 1 day" ingests that day's ledger rows, requests the cash report, recomputes every forecast and raises alerts (new HIGH or WATCH status, missing reports, gaps over 10%). The admin can also jump to a date or auto-play. A demo agent (A01) has its report held pending so the form can be filled in live.

Frontend

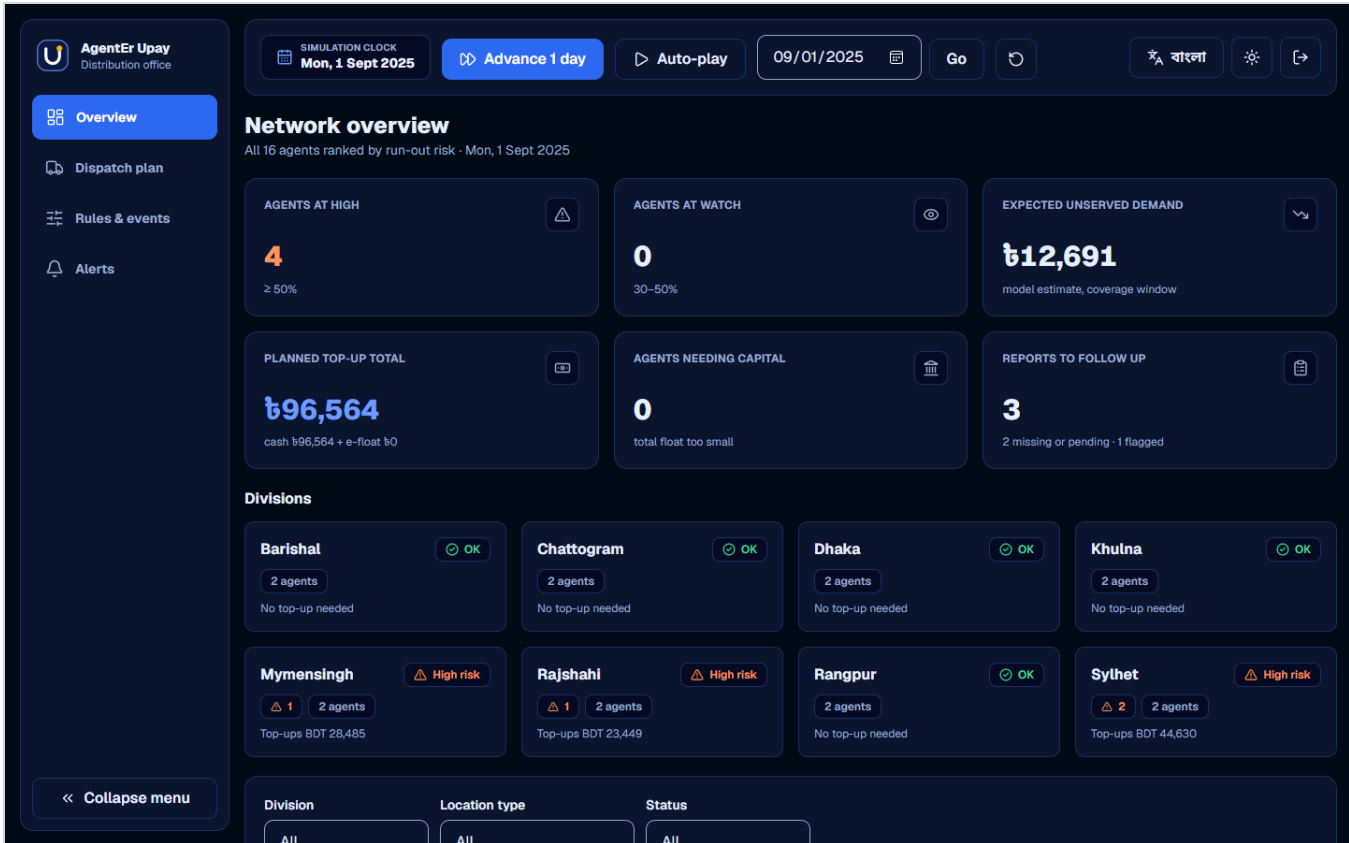
Next.js (App Router) with TypeScript, Tailwind, Recharts and Framer Motion. The design is flat (solid surfaces, 1px borders), uses the theme colour #0D1C42, has light and dark themes, focus rings, 44px touch targets and reduced-motion support. Status is always shown with colour, icon and text. Charts use a colour-blind-checked blue and orange pair. English and Bangla dictionaries are type-checked for parity, and the language button also switches digits (Bengali digits in Bangla).



Admin overview (light)



Agent home (mobile)



Admin overview (dark)

Accounts

Admin: `admin` / `admin123`. Agents sign in with their Agent ID and the password `agent` plus the lowercase id (for example Agent `A01` / `agenta01`). All 16 agents (A01 to A16) have their own home, report, forecast and history pages.

Verification

- **45 backend tests pass.** They cover login and role isolation for all 16 agents, no ground truth or generator columns outside the reveal, train/serve consistency (the serving path gives the same features and quantiles as the original code and matches the supplied `forecast_agent`), a byte-identical serving module, reconciliation edge cases, top-up timing, clock behaviour, rules, events, orders, audit log and latency budgets.
- **Frontend:** lint and a type-checked production build pass; the main flows were driven in a real browser in light, dark, English and Bangla.

5. Key features

Agent app (mobile-first)

- **Home:** two gauges (cash and e-float run-out risk) with status, the likely run-out date, balance as a share of capacity, and a "Do this next" card.
- **Daily cash report:** enter the cash count; the app reconciles it against the ledger instantly and explains the result in neutral wording.

- **7-day forecast:** demand fan charts, balance projection against a safety buffer, risk curve, per-day table, and a "Why" section with the top drivers.
- **History:** balances, top-ups and stock-out days.
- **Bangla and English** with matching digits; light and dark themes.

Admin console (desktop-first)

- **Network overview:** KPI tiles (agents at high and watch risk, model-estimated unserved demand, planned top-up total, agents needing capital, reports to follow up), division tiles and a risk-ranked, filterable agent table.
- **Agent detail:** the same forecast view plus reconciliation history, alerts and the **Demo reveal** overlay (true demand versus forecast, admin only).
- **Dispatch plan:** next working day's orders grouped by division and ordered by risk, with cash and e-float totals, planned versus emergency tags, and an order workflow (proposed, acknowledged, scheduled, done).
- **Rules and events:** edit buffers, coverage and thresholds; add local event multipliers (for example a market day or festival). Every change is validated and audit-logged, and forecasts re-run without retraining.
- **Alerts:** raised on status changes, missing reports and large gaps; acknowledged by the admin.
- **Simulation controls:** advance one day, jump to a date, auto-play, reset.

The five outputs and the reasons

#	Output	Produced by
1	Cash run-out risk % with likely run-out date	The supplied risk engine; headline is the risk before the next possible top-up
2	E-float run-out risk %	Same engine, other direction
3	Per-day cash-in/out distribution (P10-P90) and surge probability	The model's quantile predictions
4	Expected amount per day	Median quantile and Monte Carlo mean
5	Recommended inventory, gap, top-up with by-when date, extra-capital flag	Engine's recommended levels plus rules (<code>topup_by_date</code> , <code>capital_gap</code>)
+	"Why": top-5 drivers per day in BDT	Exact tree-SHAP via the supplied explanation function, mapped to Bangla/English labels

Two risk numbers are shown on purpose. Over seven days of doing nothing, risk is high for nearly everyone, so the actionable figure is the risk *before the next possible top-up*. That window lengthens before weekends and holidays, and the interface explains this in one line.

Business rules behind the recommendations

Rule	Behaviour
Status bands	Risk $\geq$ 50% HIGH, 30-50% WATCH, below 30% OK (configurable). Agent status is the worse of cash and e-float. Always colour, icon and text.
Reconciliation	E-float and flows are the ledger; physical cash is self-reported. If the report differs from the ledger by more than 10%, the entry is "Needs verification" and the ledger value is used. A missing report uses the ledger and is labelled "Estimated, not confirmed today". Wording is neutral.
Top-up by-when	Last working day before the likely run-out, never earlier than the next working day (Fridays, Saturdays and holidays are closed). If even the next working day is too late the order is flagged late.
Planned vs emergency	Emergency when risk is HIGH and run-out is on or before the next working day; otherwise planned.
Top-up meaning	Cash top-up converts e-float to cash; e-float top-up buys e-float with cash.
Capital gap	If total float is too small: extra capital = required cash + required e-float - current cash - current e-float.
Local events	An admin multiplier with a date range scales the forecast quantiles before risk simulation, labelled "manual adjustment, not learned by the model".

6. AI approach

The model

The forecaster is the supplied **LightGBM quantile model bundle**. For each agent and each of the next seven days it predicts the 10th, 25th, 50th, 75th and 90th percentiles of cash-out and cash-in demand. Inputs are built by the supplied feature code from the last 28 days of history and a calendar: weekday, month, holidays, Eid windows, payday timing, agent type and location traits, and lagged and rolling demand. The model is used exactly as supplied and is never retrained or modified by the app.

From quantiles to risk

The supplied Monte Carlo risk engine samples many demand paths from the predicted distribution, applies them to the current cash and e-float balances, and counts how often a balance falls below zero (or below a buffer). This gives the run-out risk for each day, the likely run-out date, the expected shortfall, and the inventory level that would cover a chosen share (95% by default) of scenarios. The app reuses the same sampler and seed to draw the P10, P50 and P90 balance bands, so the bands and the percentages always describe the same simulated paths.

Explanations

The supplied explanation function computes exact tree-SHAP contributions for each prediction. The app maps them to readable labels in Bangla and English and shows the top five drivers per day in taka, with wording that depends on whether the driver raises or lowers demand. Explanations are deterministic text templates: **no language model is used for any number, decision or explanation**.

Where AI stops and rules begin

Done by the model	Done by transparent rules (editable)
Daily demand quantiles; risk simulation; recommended levels; driver attribution	Status bands; cash reconciliation; top-up by-when date; planned vs emergency; capital gap; manual event multipliers

This split means a business owner can change a threshold or coverage level and see the effect immediately, and the model's behaviour never changes behind their back.

Human oversight

- Recommendations only; nothing is ordered or executed automatically.
- A local event override lets people encode knowledge the model lacks (a market day, a festival) and is labelled "manual adjustment, not learned by the model".
- Uncertainty is displayed as ranges and probabilities, and every value shows its source.
- The model knows only calendar, holiday, Eid, payday and agent-type effects. It does not know about local events unless a person adds one.

Model performance (offline, synthetic data)

Weighted absolute percentage error on the Sep to Dec test period: **17.1%** cash-out and **17.5%** cash-in, against 31.5% and 30.3% for a same-weekday-last-week baseline and a synthetic noise floor of 16.1%, so the model is close to the best achievable on this data. The stated 80% prediction interval actually covers 77.0% (cash-out) and 75.3% (cash-in). An unseen agent scores 17.2% versus 16.9% for a seen one; with Eid-ul-Adha held out of training, error rises to 29.2% (cash-out) and 22.9% (cash-in), a reminder that rare events need the manual override.

7. Intended real-life impact

The figures below come from the supplied offline evaluation notebook: a policy replay over 1 Sep to 24 Dec 2025 (16 agents, 115 days) on **synthetic** data. They show what the approach could achieve, not what it has achieved on real upay data.

Policy	Stock-out days	Unserved (BDT)	Orders
Agent habit (order below 30%, top up to 65%)	64	653,720	309
Hybrid (habit buffer raised by the model's 95% requirement) - recommended	29	232,643	503
Model only (just-in-time)	76	549,632	518

What this means in practice

- **Fewer lost transactions.** The hybrid policy cuts stock-out days by about 55% and unserved demand by about 64% compared with agent habit. For customers that means fewer failed cash-outs and cash-ins; for upay, more completed transactions and more trust.

- **An honest cost.** It needs about 63% more orders (trips). Whether that pays off depends on the cost of a trip versus an unserved customer; coverage is a business dial (80%, 90%, 95%, 99% coverage gives 40, 36, 29, 18 stock-out days for 396, 450, 503, 587 orders).
- **The model supports, not replaces, experience.** Using the model alone (just-in-time) is worse than habit (76 vs 64 stock-out days). Its value is knowing when the habit buffer is not enough.
- **Benefits differ by agent type.** Stock-out reduction: rural 78%, university 71%, market 50%, garment 20%, remittance-urban 0%.
- **It exposes a capital problem.** For garment agents the 95% requirement exceeds total float on 10.7% of days (average shortfall about BDT 13,062). No timing improvement fixes that, and the app flags it as "needs extra capital" so it can be discussed with the agent.

Intended real-life impact

- **Agents** get a daily, plain-language answer on a phone, so they can act before they run out, and a clear explanation to trust.
- **Distributors** get a ranked, grouped plan for the next working day, with urgent orders separated from routine ones, instead of reacting to calls.
- **upay** sees network-wide risk, estimated unserved demand and capital gaps, and can tune policy (coverage, thresholds) in one place.
- **Customers** are served more reliably, especially around paydays, holidays and Eid.

Path to a real pilot

1. Validate the model on governed real upay data (this prototype's accuracy is on synthetic data and in-sample for the deployed model).
2. Measure the real cost of a trip versus an unserved customer, and set the coverage level from it.
3. Replace mock sign-in and placeholder branding with upay identity and logo.
4. Pilot with a small group of agents and one distributor, with humans deciding every order.

8. Change history after the first delivery

1. Interface changed from neumorphism to **flat design**.
2. Theme colour set to `#0D1C42`; the dark theme was made darker, then re-based on a supplied reference (near-black navy page, darker navy cards, solid blue active navigation, orange for high risk).
3. The separate digit toggle was merged into the language button; digits follow the language.
4. Login changed to **Agent ID** with a matching password; all 16 agents seeded with their own pages.
5. The in-app Impact & model page and the "Synthetic data prototype" notice were removed from the interface (the evaluation results are kept in section 7 of this report and in the API).

9. Assumptions

1. No upay logo file was supplied: the logo mark is a placeholder and the warm yellow accent is assumed.
2. `agents_metadata.csv` was missing; agent traits and capacities come from the dataset's static columns. Generator internals in the bundle's agent table are dropped immediately and never used or shown.
3. Some colour tokens were adjusted from the spec to meet WCAG AA contrast for text.

4. The ledger is the dataset: orders and "mark done" are planning records and do not change replayed balances.
5. The forecast cash input is the agent's count when within tolerance, otherwise the ledger value.
6. Manual events apply to forecast target dates in their range; multiple events multiply.
7. "Model-estimated unserved demand" is the expected shortfall inside the coverage window from the same Monte Carlo paths, never ground truth.
8. The calendar covers 2025 only, so forecasts in the last week of December have no holiday information for early January 2026.
9. Authentication is a demo (mock users, development JWT secret).
10. Docker and compose files are provided but were not run, as Docker was not installed on the build machine; the local virtual-environment and npm path was verified end to end.

10. Limitations

- Synthetic data: the model recovers planted patterns, so results validate the pipeline, not real-world accuracy.
- One year of data only, and the deployed model was trained on all of 2025, so replay dates are in-sample.
- The risk engine assumes independent days, so tail risk is probably understated.
- Recommendations only; nothing is autonomous.
- **Next steps:** validate on governed upay data before any pilot; replace the mock auth and placeholder branding; confirm the cost of a trip versus an unserved customer to set the coverage level; run the Docker path in CI.